

JAVA

# Testes Unitários com JUnit

Da primeira asserção aos testes parametrizados

---

Apostila completa sobre o framework JUnit 4, com exemplos práticos e explicações passo a passo de cada anotação e cada boa prática.

## CONTEÚDO DA APOSTILA

- 01 Por Que Testar
- 02 A Primeira Classe de Teste
- 03 Stubs e a Primeira Asserção
- 04 Independência dos Testes
- 05 Boas Práticas de Nomenclatura
- 06 Testando Exceções Esperadas
- 07 Outros Métodos de Asserção
- 08 Setup e Teardown com @Before e @After
- 09 Configuração Única com @BeforeClass
- 10 Testes Parametrizados
- 11 Boas Práticas Finais

## 01

## Por Que Testar

Testes unitários e o papel do JUnit no dia a dia

### A primeira linha de defesa

Entre as fases do ciclo de vida de uma aplicação, os testes ocupam um lugar central — e o teste unitário é a primeira linha de defesa contra bugs, normalmente escrito pelo próprio time de desenvolvimento. Em Java, uma “unidade” costuma corresponder a um único método: testar um método significa verificar, de forma isolada, se ele devolve o resultado esperado para um determinado conjunto de entradas.

É perfeitamente possível testar um método manualmente — chamando-o algumas vezes com valores diferentes e conferindo o resultado no console. O problema aparece na segunda rodada: qualquer alteração no código exige repetir tudo de novo, e é fácil esquecer algum caso já coberto antes.

Um framework de testes unitários resolve isso automatizando a execução. O mais usado em projetos Java é o JUnit, e a versão 4 — a que este material usa do início ao fim — trouxe anotações que simplificaram bastante a escrita de testes em relação às versões anteriores.

### O domínio deste material: ContaZen

Para exercitar o JUnit na prática, vamos construir e testar uma pequena carteira digital fictícia chamada ContaZen, com operações de depósito e saque, sujeitas a uma regra de negócio simples: saques acima de determinado valor só são permitidos quando feitos através de uma agência, e não por um caixa eletrônico.

```
public class ContaDigital {

    public static final int TIPO_CORRENTE = 1;
    public static final int TIPO_POUPANCA = 2;

    private String titular;
    private double saldo;
    private int tipoConta;

    public ContaDigital(String titular, double saldo, int tipoConta) {
        this.titular = titular;
        this.saldo = saldo;
        this.tipoConta = tipoConta;
    }

    public double depositar(double valor, boolean viaAgencia) {
        saldo += valor;
        return saldo;
    }

    public double getSaldo() {
        return saldo;
    }

    public boolean isContaCorrente() {
        return tipoConta == TIPO_CORRENTE;
    }
}
```

O método de saque, com a regra de limite, vai entrar mais adiante — no capítulo sobre exceções esperadas. Por ora, essa base já é suficiente para escrever os primeiros testes.

## 02

## A Primeira Classe de Teste

Criando a estrutura e configurando a biblioteca JUnit 4

### Gerando a classe de teste

O IntelliJ oferece um atalho para criar a classe de teste correspondente a uma classe já existente: com o cursor na linha de declaração da classe — `public class ContaDigital` —, o atalho `Alt+Enter` (`Option+Enter` no macOS) abre a opção `Create Test`.

Por convenção, a classe de teste recebe o mesmo nome da classe original acrescido do sufixo `Test` — neste caso, `ContaDigitalTest` — e fica no mesmo pacote da classe testada, embora normalmente em uma pasta de código-fonte separada, dedicada a testes. Isso evita que código de teste acabe empacotado junto com a aplicação final.

Na mesma janela, é possível marcar quais métodos já devem ganhar um método de teste correspondente. Marcando todos, o IntelliJ gera um stub — um método vazio, anotado — para cada um.

```
public class ContaDigitalTest {  
  
    @org.junit.Test  
    public void depositar() {  
    }  
  
    @org.junit.Test  
    public void getSaldo() {  
    }  
}
```

### Adicionando a biblioteca JUnit 4 corretamente

O IntelliJ traz o JUnit embutido na própria distribuição, mas ele não é adicionado automaticamente a um projeto novo. Ao criar o primeiro teste, a IDE costuma avisar que a biblioteca não foi encontrada e oferece um botão para corrigir isso — só que, se essa correção for aceita sem ajuste, o JUnit costuma ser adicionado com escopo de teste apenas, e as anotações continuam aparecendo como não resolvidas na hora de compilar.

**ATENÇÃO — Escopo da biblioteca: Compile, não apenas Test**

Se as anotações do JUnit continuarem sublinhadas mesmo depois de adicionar a biblioteca, abra File › Project Structure › Modules, localize a entrada da biblioteca JUnit e mude seu escopo para Compile. Alternativamente, ao corrigir o erro direto pelo Alt+Enter sobre a anotação não resolvida, o IntelliJ já aplica o escopo correto de uma vez, sem precisar passar por essa tela.

Por fim, vale criar uma configuração de execução dedicada à classe de teste: com o cursor fora de qualquer método, um clique com o botão direito sobre a classe oferece a opção Run ContaDigitalTest, que passa a rodar todos os métodos de teste da classe de uma só vez.

## 03

## Stubs e a Primeira Asserção

De métodos que sempre falham a comparações reais com assertEquals

### Por que stubs vazios são perigosos

Um método de teste vazio, sem nenhuma verificação, é reportado pelo JUnit como aprovado — afinal, nenhuma condição foi violada. O problema é que isso passa uma falsa sensação de segurança: nada ali está realmente testando coisa alguma.

A prática recomendada é chamar o método `fail()` dentro de cada stub recém-criado, com uma mensagem lembrando que o teste ainda não foi implementado. Assim, a suíte de testes falha até que o teste seja de fato escrito — eliminando o risco de esquecer um stub vazio disfarçado de teste aprovado.

```
@org.junit.Test
public void depositar() {
    org.junit.Assert.fail("Teste ainda não implementado");
}
```

### A primeira asserção de verdade

Substituindo o stub por um teste real, a ideia é criar uma conta, chamar `depositar` e comparar o saldo resultante com o valor esperado, usando `assertEquals`. O primeiro parâmetro é o valor esperado; o segundo é o valor obtido de fato.

```
@org.junit.Test
public void depositar() {
    ContaDigital conta = new ContaDigital("Marina", 1000.00,
        ContaDigital.TIPO_CORRENTE);
    double saldo = conta.depositar(200, true);
    org.junit.Assert.assertEquals(1200.00, saldo, 0);
}
```

Repare no terceiro argumento, `0`: ao comparar valores do tipo `double`, o JUnit exige um parâmetro extra chamado delta, que define a margem de tolerância aceitável entre o valor esperado e o valor real. Isso existe porque ponto flutuante nem sempre representa valores decimais de forma exata — um resultado como 1325.14 pode chegar como 1325.1399999999999 depois de algumas operações. Começar com delta zero e só

aumentar se necessário é a abordagem mais segura.

**DICA — assertEquals compara com equals()**

Ao comparar objetos (não tipos primitivos), assertEquals utiliza o método equals() das classes envolvidas para decidir se os valores são iguais — vale garantir que esse método esteja implementado corretamente na classe testada.

## 04

## Independência dos Testes

Cada teste começando do zero, e como rodar apenas um deles

### Cada teste começa do zero

Uma regra que vale para qualquer suíte de testes: o resultado de um método de teste nunca deve depender do que aconteceu em outro. Cada teste precisa poder rodar sozinho, de forma isolada, e chegar ao mesmo resultado independentemente da ordem em que os outros testes da classe forem executados.

Isso significa, por exemplo, não guardar o estado de uma conta numa variável de instância compartilhada entre métodos de teste sem cuidado — cada teste precisa da sua própria instância de `ContaDigital`, criada do zero. (No capítulo sobre `@Before`, veremos uma forma organizada de fazer isso sem repetir a mesma linha em todo método.)

### Rodando um único método

Além de rodar a classe inteira, o IntelliJ permite executar um teste isoladamente: uma seta verde aparece na margem do editor, ao lado da assinatura de cada método anotado com `@Test`. Clicar nela roda somente aquele teste.

Cada execução individual gera sua própria configuração de execução, acessível depois pelo menu suspenso no canto superior direito da IDE — útil quando se está depurando repetidamente um teste específico, sem querer rodar a suíte inteira a cada tentativa.

#### REGRA — Isolamento não é opcional

Um teste que só passa quando executado depois de outro específico é, na prática, um teste quebrado — mesmo que apareça como aprovado na maioria das execuções. Esse tipo de dependência escondida costuma aparecer justamente quando alguém tenta rodar um teste isolado, como descrito acima, e ele falha sem motivo aparente.

## 05

## Boas Práticas de Nomenclatura

Um assert por teste, e nomes que contam a própria história

### Um assert por método de teste

É tecnicamente possível colocar várias chamadas de assert dentro do mesmo método de teste. Na prática, porém, isso dificulta o diagnóstico: quando o teste falha, fica menos óbvio qual das várias condições foi a responsável. A prática recomendada é reservar cada método de teste para uma única condição — uma única pergunta que ele responde.

Isso costuma significar dividir um teste “grande demais” em vários menores. Um teste de depósito que também conferia o saldo da conta depois da operação, por exemplo, vira dois métodos: um específico para o valor devolvido pelo método, e outro específico para o estado da conta depois da chamada.

```
@org.junit.Test
public void depositar() {
    ContaDigital conta = new ContaDigital("Marina", 1000.00,
        ContaDigital.TIPO_CORRENTE);
    double saldo = conta.depositar(200, true);
    org.junit.Assert.assertEquals(1200.00, saldo, 0);
}

@org.junit.Test
public void getSaldo_apos_deposito() {
    ContaDigital conta = new ContaDigital("Marina", 1000.00,
        ContaDigital.TIPO_CORRENTE);
    conta.depositar(200, true);
    org.junit.Assert.assertEquals(1200.00, conta.getSaldo(), 0);
}
```

### Nomes que contam a própria história

Repare no nome do segundo método: `getSaldo_apos_deposito`. Nomes de teste descritivos ajudam bastante na hora de ler o resultado de uma suíte inteira — a lista de métodos que passaram ou falharam já indica, sem abrir o código, exatamente o que cada teste estava verificando.

Essa convenção vale a pena inclusive para testes muito parecidos entre si: um teste de saque dentro do limite e outro de saque acima do limite, por exemplo, ficam bem mais claros com nomes como `sacar_dentroDoLimite` e `sacar_acimaDoLimite_semAgencia` do que com nomes genéricos como `teste1` e `teste2`.

## 06

## Testando Exceções Esperadas

Ensinando o JUnit a esperar que algo dê errado

### A regra de negócio: limite de saque

Chegou a hora de implementar o método de saque da ContaZen, com a regra mencionada no primeiro capítulo: valores acima de R\$ 500 só podem ser sacados através de uma agência — fora dela, a operação deve lançar uma exceção.

```
public double sacar(double valor, boolean viaAgencia) {
    if (valor > 500 && !viaAgencia) {
        throw new IllegalArgumentException(
            "Saque acima do limite exige atendimento em agência");
    }
    saldo -= valor;
    return saldo;
}
```

### Ensinando o teste a esperar uma exceção

Testar esse cenário exige uma abordagem diferente: o teste só deve passar se a exceção for de fato lançada. Isso é feito informando a exceção esperada como um atributo da própria anotação `@Test`, em vez de usar asserções dentro do corpo do método.

```
@org.junit.Test(expected = IllegalArgumentException.class)
public void sacar_acimaDoLimite_semAgencia() {
    ContaDigital conta = new ContaDigital("Marina", 1000.00,
        ContaDigital.TIPO_CORRENTE);
    conta.sacar(600, false);
}
```

Se a exceção informada em `expected` for lançada durante a execução do método, o JUnit considera o teste aprovado. Se ela não for lançada — ou se uma exceção de outro tipo for lançada em seu lugar —, o teste falha.

**DICA — A forma antiga: try/catch vazio**

Antes da anotação `expected` existir, o JUnit exigia envolver o código em um bloco `try/catch`, deixando o `catch` vazio (já que nenhuma asserção precisava ser feita). Isso ainda funciona, mas a versão com `expected` é bem mais direta e é a abordagem recomendada em qualquer código novo.

Vale reforçar: quando o teste espera uma exceção, não faz sentido combinar isso com outro `assert` no mesmo método verificando, por exemplo, o saldo — se o método lançou a exceção, a linha de `assert` nunca seria alcançada de qualquer forma.

## 07

## Outros Métodos de Asserção

assertArrayEquals, assertNull, assertSame e assertThat

### Além do assertEquals

A classe `org.junit.Assert` oferece vários métodos de asserção além do já conhecido `assertEquals`, cada um pensado para deixar a intenção do teste mais clara.

| Método                                               | Quando usar                                                      |
|------------------------------------------------------|------------------------------------------------------------------|
| <code>assertNotEquals</code>                         | Quando o valor atual NÃO deve ser igual ao esperado              |
| <code>assertArrayEquals</code>                       | Para comparar o conteúdo de dois arrays, elemento a elemento     |
| <code>assertNull</code> / <code>assertNotNull</code> | Para verificar se um valor é (ou não é) nulo                     |
| <code>assertSame</code> / <code>assertNotSame</code> | Para verificar se duas referências apontam para o mesmo objeto   |
| <code>assertThat</code>                              | Para comparar um valor contra um matcher, com mais flexibilidade |

### Por que assertEquals existe

Comparar dois arrays com `assertEquals` quase nunca faz o que se espera: esse método só considera dois arrays iguais se forem exatamente a mesma instância na memória, e não se tiverem o mesmo conteúdo. `assertArrayEquals` resolve isso comparando o comprimento e cada elemento, na mesma ordem.

```
@org.junit.Test
public void extrairDigitosVerificadores() {
    char[] esperado = {'4', '2'};
    char[] resultado = ContaDigital.extrairDigitos("CZ-4291-2");
    org.junit.Assert.assertArrayEquals(esperado, resultado);
}
```

### assertSame e a diferença entre igual e idêntico

Enquanto `assertEquals` usa o método `equals` para decidir se dois objetos são “iguais” (mesmo conteúdo), `assertSame` compara apenas as referências — só passa se as duas variáveis apontarem exatamente para o mesmo objeto na memória. É a ferramenta certa para confirmar, por exemplo, que um método devolveu a própria instância recebida, em vez de criar uma cópia.

## 08

## Setup e Teardown com @Before e @After

Preparando e limpando o cenário a cada teste

### Eliminando repetição com @Before

Praticamente todo teste da ContaZen começa criando uma nova instância de `ContaDigital` com os mesmos valores. Repetir essa linha em cada método funciona, mas é código redundante — e qualquer mudança na assinatura do construtor exigiria atualizar todos os testes um por um.

A anotação `@org.junit.Before` resolve isso: um método marcado com ela roda automaticamente antes de cada teste da classe, sempre com uma instância nova. O padrão é guardar o objeto recém-criado em uma variável de instância da própria classe de teste, para que os métodos de teste possam acessá-lo.

```
public class ContaDigitalTest {

    private ContaDigital conta;

    @org.junit.Before
    public void setUp() {
        conta = new ContaDigital("Marina", 1000.00,
            ContaDigital.TIPO_CORRENTE);
    }

    @org.junit.Test
    public void depositar() {
        double saldo = conta.depositar(200, true);
        org.junit.Assert.assertEquals(1200.00, saldo, 0);
    }

    @org.junit.Test
    public void sacar_dentroDoLimite() {
        double saldo = conta.sacar(300, false);
        org.junit.Assert.assertEquals(700.00, saldo, 0);
    }
}
```

Repare que, mesmo os dois testes rodando sobre a mesma variável `conta`, o isolamento continua garantido: o método marcado com `@Before` roda de novo — criando uma conta nova — antes de cada um deles, então

nenhum teste enxerga o efeito colateral do outro.

## Limpando depois com @After

De forma simétrica, `@org.junit.After` marca um método que roda automaticamente depois de cada teste — útil para liberar recursos abertos durante o teste, como uma conexão simulada ou um arquivo temporário.

```
@org.junit.After
public void tearDown() {
    conta = null;
}
```

## 09

## Configuração Única com @BeforeClass

Código que roda uma vez só, antes de todos os testes

### Quando @Before não é suficiente

Existem cenários em que a configuração não deve se repetir a cada teste, mas rodar uma única vez por execução da suíte inteira — por exemplo, abrir uma conexão simulada com um serviço externo, ou carregar um conjunto de dados fixo que todos os testes vão consultar. Repetir isso a cada teste seria desnecessário e mais lento.

Para esse caso existem `@org.junit.BeforeClass` e `@org.junit.AfterClass`. O primeiro roda uma única vez, antes de qualquer teste da classe; o segundo roda uma única vez, depois que todos os testes já terminaram. Diferente de `@Before` e `@After`, esses métodos precisam ser declarados como `static`.

```
@org.junit.BeforeClass
public static void conectarServicoDeCambio() {
    System.out.println("Conectando ao serviço de câmbio (uma vez)");
}

@org.junit.AfterClass
public static void encerrarServicoDeCambio() {
    System.out.println("Encerrando conexão (uma vez)");
}
```

#### ATENÇÃO — A ordem no console pode enganar

Instruções de impressão dentro de métodos `@BeforeClass`, `@Before`, `@Test`, `@After` e `@AfterClass` costumam ficar em uma fila de saída (thread de I/O), então a ordem em que as mensagens aparecem no console nem sempre reflete perfeitamente a ordem real de execução. A ordem de execução dos métodos, porém, segue sempre a mesma lógica: `@BeforeClass` primeiro, `@AfterClass` por último, com `@Before/@Test/@After` intercalados para cada teste no meio do caminho.

### Um contador simples para confirmar a ordem

Para verificar isso na prática, um campo estático incrementado em cada método — e impresso antes do incremento — mostra a sequência exata: o valor exibido por `@BeforeClass` é sempre zero, os testes seguem em ordem crescente a partir dali, e o valor final aparece no `@AfterClass`, confirmando que ele realmente roda

por último, mesmo quando o console mostra as linhas fora dessa ordem.

## 10

## Testes Parametrizados

Um único método de teste para vários conjuntos de dados

### O problema: cinco testes quase idênticos

Imagine testar o método `depositar` com cinco combinações diferentes de valor depositado e saldo esperado. Escrever cinco métodos de teste separados funciona, mas é repetitivo — a estrutura do teste é sempre a mesma, só os números mudam.

O JUnit resolve isso com testes parametrizados: uma classe de teste especial, marcada com `@RunWith(Parameterized.class)`, para a qual o framework cria uma nova instância a cada conjunto de dados, passando os valores pelo construtor.

### Montando a classe parametrizada

A classe precisa de: um método estático anotado com `@Parameters`, devolvendo uma coleção de arrays de objetos — cada array representando um conjunto de dados de teste; variáveis de instância para guardar cada parâmetro; e um construtor que recebe e atribui esses valores.

```
@org.junit.runner.RunWith(org.junit.runners.Parameterized.class)
public class ContaDigitalDepositoParametrizadoTest {

    private double valorDepositado;
    private double saldoEsperado;

    public ContaDigitalDepositoParametrizadoTest(
        double valorDepositado, double saldoEsperado) {
        this.valorDepositado = valorDepositado;
        this.saldoEsperado = saldoEsperado;
    }

    @org.junit.runners.Parameterized.Parameters
    public static java.util.Collection<Object[]> condicoesDeTeste() {
        return java.util.Arrays.asList(new Object[][] {
            {100.00, 1100.00},
            {200.00, 1200.00},
            {325.14, 1325.14},
            {489.33, 1489.33},
            {1000.00, 2000.00},
        });
    }

    private ContaDigital conta;

    @org.junit.Before
    public void setUp() {
        conta = new ContaDigital("Marina", 1000.00,
            ContaDigital.TIPO_CORRENTE);
    }

    @org.junit.Test
    public void depositar() {
        double saldo = conta.depositar(valorDepositado, true);
        org.junit.Assert.assertEquals(saldoEsperado, saldo, 0.01);
    }
}
```

Rodando essa classe, o JUnit executa o método `depositar` cinco vezes — uma para cada linha da coleção devolvida por `condicoesDeTeste` — e reporta cada execução separadamente, identificada pelo índice do

conjunto de dados usado.

**DICA — Delta maior para evitar falsos negativos**

Repare que o delta usado aqui é 0.01, e não zero: ao somar valores double repetidas vezes, pequenos erros de arredondamento se acumulam, e um delta zero acabaria reprovando um teste que, na prática, está correto.

## 11

## Boas Práticas Finais

Reduzindo repetição e organizando a suíte de testes

### Um teste por classe testada

Assim como cada método de teste deveria verificar uma única condição, é boa prática manter uma classe de teste dedicada por classe de produção — em vez de espalhar testes de **ContaDigital** por várias classes diferentes sem critério. Isso mantém a suíte organizada e fácil de navegar conforme o projeto cresce.

### Revisando o que este material cobriu

| Recurso                                 | Para que serve                                    |
|-----------------------------------------|---------------------------------------------------|
| @Test                                   | Marca um método como um caso de teste             |
| assertEquals / assertTrue / assertFalse | Comparações básicas de valores                    |
| @Test(expected = ...)                   | Espera que uma exceção específica seja lançada    |
| @Before / @After                        | Roda antes/depois de CADA teste                   |
| @BeforeClass / @AfterClass              | Roda uma única vez para a classe inteira          |
| @RunWith(Parameterized.class)           | Um método de teste para vários conjuntos de dados |

### Cuidado com falsos positivos

Vale fechar reforçando o alerta mais importante deste material: um teste só é útil na medida das asserções que ele realmente faz. Um stub vazio, um método sem **fail()**, ou uma condição que nunca chega a ser testada de verdade — todos esses casos aparecem como “aprovados” no relatório do JUnit, mesmo sem garantir nada sobre o comportamento real do código.

Ler os nomes dos métodos de teste depois de rodar a suíte, revisando rapidamente se cada nome realmente corresponde a uma verificação significativa, é um hábito simples que evita esse tipo de falsa segurança — e vale manter mesmo em uma suíte que já está toda verde.

## Fim de apostila

Este capítulo consolidou o essencial do JUnit 4: asserções, exceções esperadas, ciclo de vida de teste com `@Before/@After` e `@BeforeClass/@AfterClass`, e testes parametrizados.

Esses fundamentos sustentam qualquer suíte de testes Java daqui em diante — o próximo passo natural é explorar frameworks de mock para isolar dependências externas.